AZ-305 · D2 — Data Storage

SHAPE + ACCESS · smallest store that meets SLA · Session default · Columnstore = S3+ · RA-GZRS = read 2nd + zone-safe

SQL family selection

OS access, SQL Agent cross-DB, FileStream, CLR, SSAS/SSRS on box, BYOK HSM → **SQL on VM (IaaS)**

Most SQL features (cross-DB, Agent, Broker, linked servers) + PaaS + VNet → **SQL MI**

Single-DB / SaaS multi-tenant, max PaaS abstraction → **SQL Database**

SQL MI = **delegated subnet /27**. Enterprise / Always On AG lift-shift → **Business Critical** (built-in HA).

Azure SQL DB tier

DTU model (bundled compute+storage+IO)

- Dev/test → **Basic (5)**
- Most OLTP (Columnstore = S3+) → **Standard**
- High IO, in-mem OLTP, sub-2ms, read scale-out → **Premium**

vCore model (independent scale + reservations)

- Most workloads → **General Purpose**
- Local SSD, in-mem OLTP, +1 free readable replica → **Business Critical**
- >4 TB, near-instant backups, 4 read replicas → **Hyperscale**

DTU floor ≈ IOPS ÷ 3, round **up**. Hyperscale triggers: "100 TB", "near-instant scaling/restore".

Cosmos DB API selection

SQL queries + relational tables + geo-replication → **PostgreSQL**

Document JSON, greenfield (flagship) → **NoSQL / Core**

Existing MongoDB app → **MongoDB**

Existing Cassandra app → **Cassandra**

Existing Gremlin graph app → **Gremlin**

Azure Table app + global + autoscale → **Table**

Cosmos is the storage answer for SLA on **read/write latency + throughput + consistency + availability**.

Right data → right store

PostgreSQL: 1 node vs sharded

Fits on one node (vertical scale + read replicas)

- OLTP · web/app backends · single / low-tenant SaaS → **Flexible Server**

Outgrew one node → shard writes/storage

- Multi-tenant SaaS (shard by tenant_id) · real-time analytics · high-ingest time-series → **Cosmos for PostgreSQL (Citus)**

Not a Cosmos API pick — **vanilla PostgreSQL**. Citus only after vertical scale + read replicas run out.

Cosmos consistency level

Never stale, any latency cost OK → **Strong**

Bounded lag (time/versions), cheaper than Strong → **Bounded Staleness**

Read-your-own-writes per user (web) — DEFAULT → **Session**

Write-order reads, freshness unbounded → **Consistent Prefix**

No order, cheapest, lowest latency → **Eventual**

"Sober Bartenders Serve Cold Eggnog" · Multi-write → Session or weaker.

Storage account type

General-purpose, all tiers, modern (default) → **StorageV2**

High-throughput block blob, low-latency → **BlockBlob (Premium)**

ADLS Gen2, big-data analytics (HNS) → **V2 + HNS**

NFS/SMB shares, low latency → **FileStorage (Premium)**

Classic block+page only → **V1 (don't pick)**

Storage ABAC: RBAC + condition on resource/request/principal attrs. Blobs/ADLS + Queues ✅; Files/Tables ❌.

Immutable blob (WORM)

Fixed compliance window (SEC 17a-4) → **Time-based retention**

Open-ended litigation / audit hold → **Legal hold**

Locked = compliance state of time-based retention (owner can't shorten).

Blob access tier + lifecycle

Tier	Min	Use
Hot	none	Frequent
Cool	30 d	Infrequent, instant
Cold	90 d	Rare ~90d+, instant
Archive	180 d	Rare, **rehydrate hrs**

"Need immediate access" → **never Archive**. "Automatically tier / based on age or last access / no code" → **Lifecycle Mgmt policy** (runs ~daily, free; not Functions, not Data Factory). React the instant a blob lands → Event Grid.

Storage redundancy

Type	Survives	Read 2nd?
LRS	Disk failure	—
ZRS	Zone or data center failure	—
GRS	Region	No
GZRS	Region + AZ	No
RA-GRS	Region	**Yes**
RA-GZRS	Region + AZ	**Yes**

"Read 2nd w/o failover" → **RA-GRS / RA-GZRS**. Ladder: LRS<ZRS<GRS<GZRS; survive a zone → ZRS/GZRS; zone+geo → **GZRS family**.

Data ingestion / integration

Scheduled batch ETL, many sources → **Data Factory**

Streaming events + windowed aggregation → **Stream Analytics**

React to discrete resource event (push) → **Event Grid**

Massive append-only stream (M/sec), Kafka → **Event Hubs**

Enterprise msg: FIFO, dead-letter, sessions → **Service Bus**

Synapse component

MPP enterprise data warehouse → **Dedicated SQL**

Ad-hoc query over lake, pay-per-TB → **Serverless SQL**

Notebooks · Spark · ML → **Spark pool**

No-ETL near-real-time from Cosmos/SQL → **Synapse Link**